《现代软件工程课程设计》中期报告

小组编号	08	
小组成员	学号	姓名
	2024141460123	陈琦毅
	2024141460329	杨若随

提交报告时间：　　2026 年　 11 月　 4 日

目 录

1 已实现功能.....1
2 建模与构建.....1
2.1 功能 代码量统计与可视化报告生成.....1
2.1.1 场景建模.....1
2.1.2 类建模.....2
2.1.3 功能建模/行为建模.....3
2.1.4 代码结构及说明.....5
2.1.5 界面截图.....7
2.2 功能 小学生口算出题.....10
2.2.1 场景建模.....10
2.2.2 类建模.....10
2.2.3 功能建模/行为建模.....12
2.2.4 代码结构及说明.....13
2.2.5 界面截图.....14
2.3 功能 小组项目主页展示.....16
2.3.1 场景建模.....16
2.3.2 类建模.....17
2.3.3 功能建模/行为建模.....17
2.3.4 代码结构及说明.....19
2.3.5 界面截图.....20

1 已实现功能

小组目前已实现的功能，如表 1.1 所示。

表 1.1 已实现功能列表

	功能	责任人
统计代码量	多语言代码文件递归扫描与扩展名自动识别多语言代码文件递归扫描与扩展名自动识别	陈琦毅
	有效代码行、空行、纯注释行、函数个数精准统计	陈琦毅
	函数长度多维度指标计算（最大值 / 最小值 / 均值 / 中位数）	陈琦毅
	HTML 可视化统计报告自动生成（含柱状图与分布图）	陈琦毅
	汇总表 + 明细表双维度 CSV 数据导出	陈琦毅
出题	**出题练习**：可自定义题型、难度，自动批改、计时，支持打印试卷	杨若随
	趣味对战：双人 PK 竞速答题，增加练习趣味性	杨若随
	错题记录：自动留存错题、练习历史，方便复盘巩固	杨若随
	激励体系：星星奖励、成就解锁，正向鼓励练习	杨若随
个人/小组主页	项目背景介绍与核心功能模块可视化展示	陈琦毅
	小组成员信息公示与分工职责说明	陈琦毅
	项目文档、工具源码与在线体验入口集中分发	陈琦毅
	基于 GitHub Pages 的静态页面部署与持续更新维护	陈琦毅
其他		

2 建模与构建

2.1 功能 代码量统计与可视化报告生成

2.1.1 场景建模

用例名称：代码量统计与可视化报告生成
参与者：开发人员（用户）

上下午目标： 用户选定本地代码项目目录后，系统自动完成多维度代码量统计，生成可视化 HTML 报告与 CSV 数据文件，直观呈现项目代码规模、语言分布与函数质量指标。

前置条件：

1. 本地存在待统计的代码项目文件夹
2. 运行环境具备目标文件夹的读取权限
3. 设备已安装 Python 3.6 及以上版本

后置条件：

1. 生成一份 HTML 格式的可视化统计报告
2. 生成汇总 CSV 与明细 CSV 两份数据文件
3. 系统自动调用默认浏览器打开报告

主事件流：

1. 用户启动程序，系统弹出文件夹选择对话框
2. 用户选择待统计的代码项目根目录
3. 系统递归遍历目录下所有文件，按扩展名识别有效代码文件，自动过滤非代码文件与排除目录
4. 系统逐个读取有效代码文件，逐行进行词法分析：
   - 判定并统计有效代码行数、空行数、纯注释行数
   - 识别函数定义语句，记录每个函数的代码行数
5. 系统按语言维度汇总所有文件的统计数据，计算函数长度的最大值、最小值、均值、中位数四项指标
6. 系统组装 HTML 报告模板，嵌入统计数据与可视化图表
7. 系统导出语言维度汇总 CSV 与文件维度明细 CSV
8. 系统自动调用默认浏览器打开 HTML 报告，用例结束

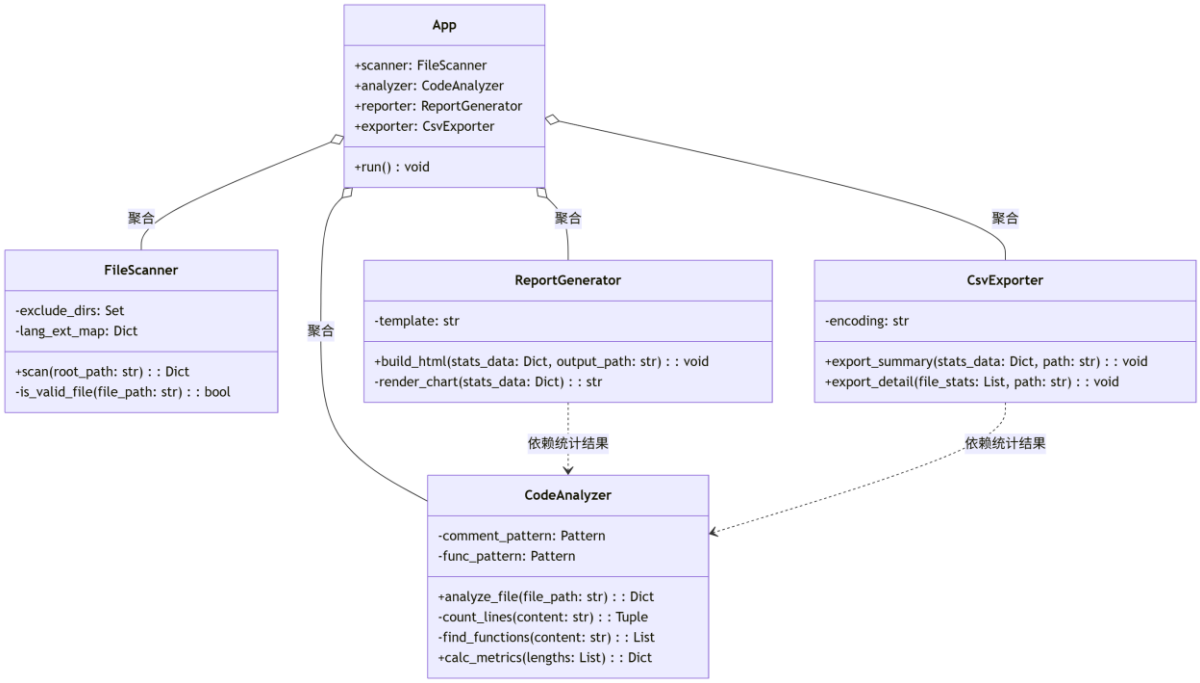
备选流：

- 若用户取消文件夹选择：程序直接退出
- 若目录内无有效代码文件：弹出提示后程序结束
- 若遇到无法读取的异常文件：跳过该文件并记录，继续统计其余文件

频率：项目开发阶段每周 1~2 次；课程作业交付、版本迭代时集中使用 1 次

2.1.2 类建模

采用面向对象思想进行类设计，遵循单一职责原则，共设计 5 个核心类，通过聚合与依赖关系协作完成全流程。

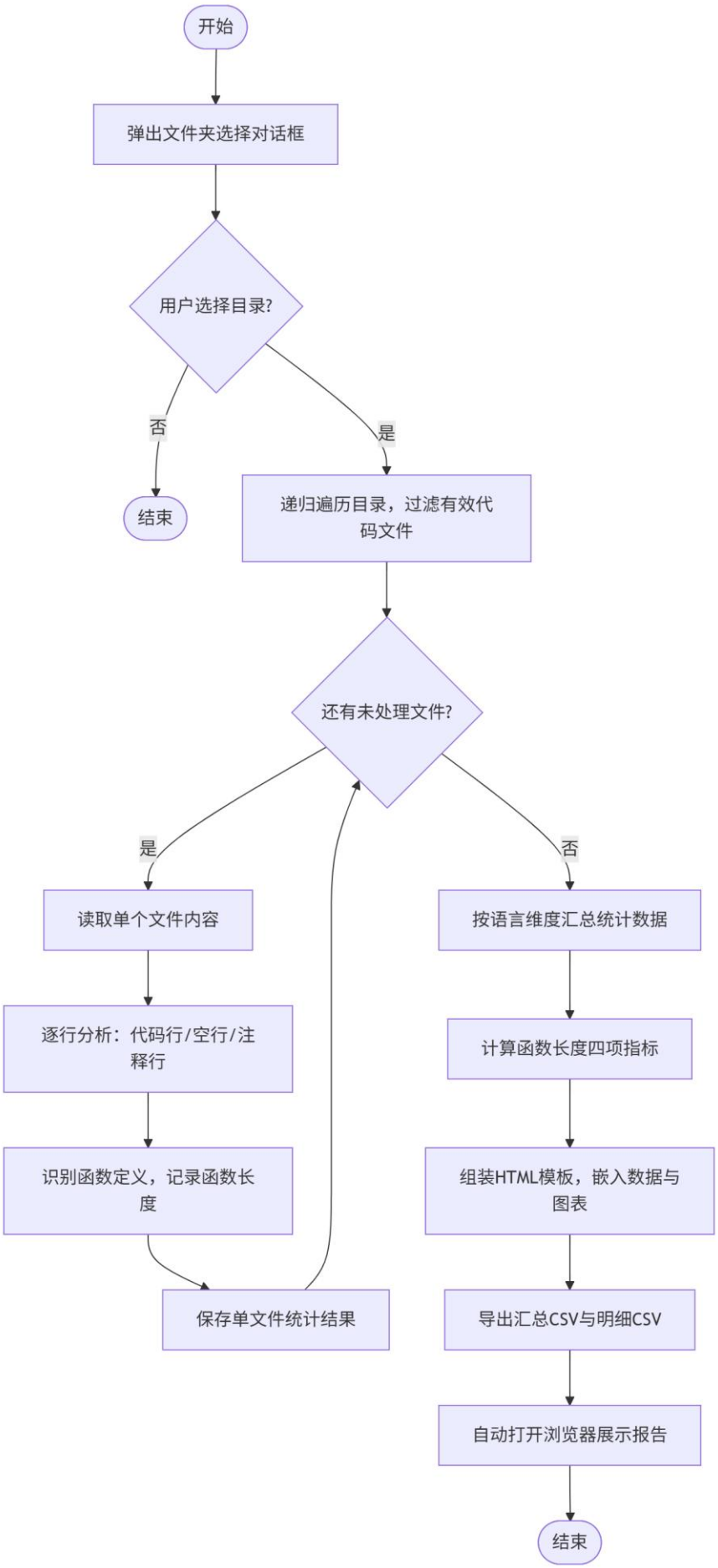


- **聚合关系**：App 作为主控制类，聚合了扫描、分析、报告、导出四个功能模块，整体与部分的生命周期相互独立
- **依赖关系**：报告生成与 CSV 导出模块均依赖代码分析模块输出的结构化统计数据，无直接属性关联

2.1.3 功能建模/行为建模

（1）活动图

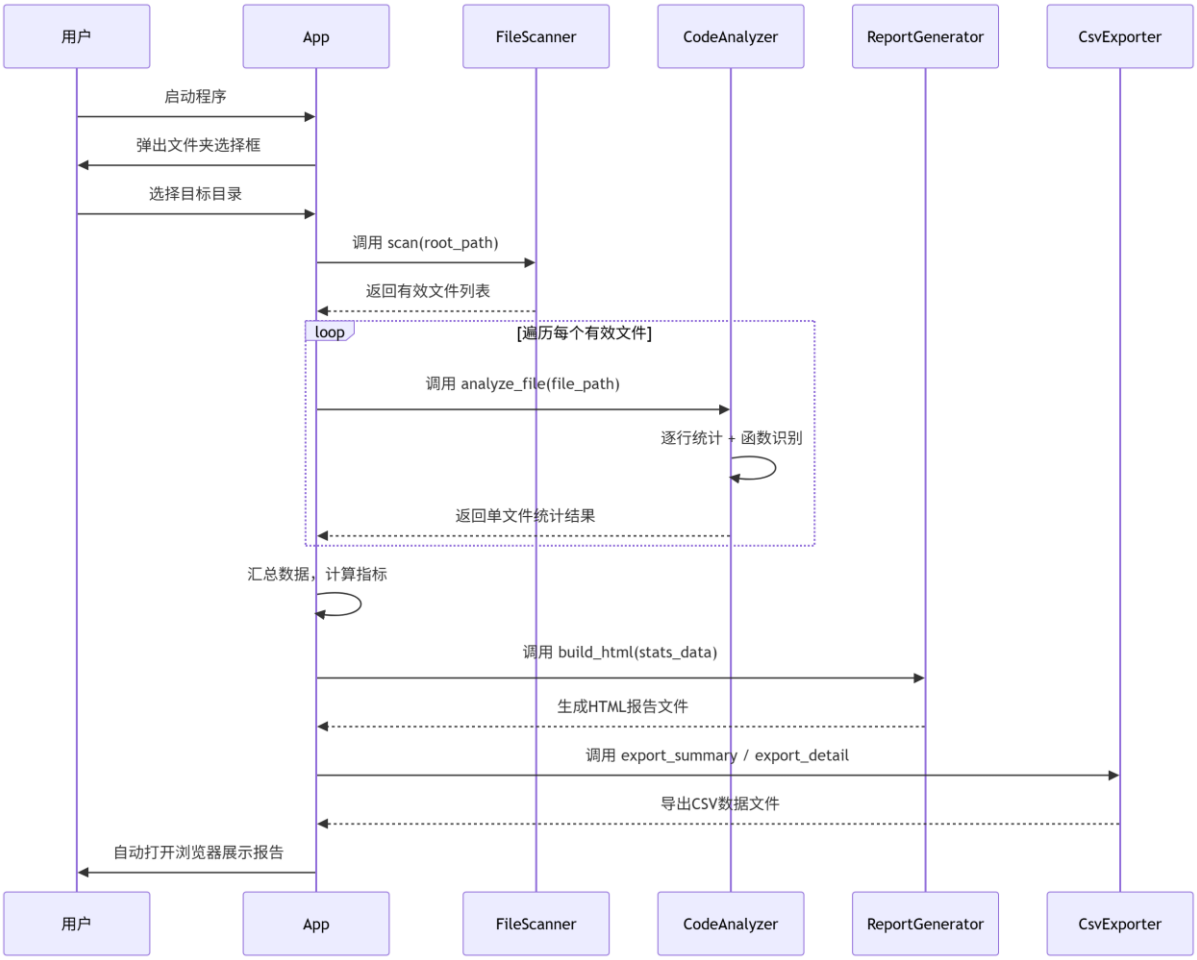
描述代码量统计的完整执行流程，覆盖主事件流与分支判断。


开始
弹出文件夹选择对话框
用户选择目录?
否
是
结束
递归遍历目录，过滤有效代码文件
还有未处理文件?
是
否
读取单个文件内容
逐行分析：代码行/空行/注释行
识别函数定义，记录函数长度
保存单文件统计结果
按语言维度汇总统计数据
计算函数长度四项指标
组装HTML模板，嵌入数据与图表
导出汇总CSV与明细CSV
自动打开浏览器展示报告
结束

（2）顺序图

描述统计流程中各对象之间的消息交互顺序，体现模块间的调用关系。

顺序图（Mermaid 源码）：



2.1.4 代码结构及说明

项目采用单文件模块化分层设计，核心结构如下：

code_stats_report.py

├── 常量配置层

│ ├── LANG_EXT_MAP 语言与文件扩展名映射表

│ ├── EXCLUDE_DIRS 默认排除目录集合

│ └── REGEX_PATTERNS 预编译正则表达式（注释、函数识别）

│

├── FileScanner 类（目录扫描层）

```
│   ├── scan()              递归遍历目录，返回按语言分组的文件列表
│   └── is_valid_file()   判断文件是否为有效代码文件
│
├── CodeAnalyzer 类（核心分析层）
│   ├── analyze_file()    单文件统计入口
│   ├── count_lines()     逐行判定代码/空行/注释
│   ├── find_functions()  正则匹配函数定义
│   └── calc_metrics()    计算函数长度四项指标
│
├── ReportGenerator 类（展示层）
│   └── build_html()      组装 HTML 模板，嵌入数据与图表
│
├── CsvExporter 类（导出层）
│   ├── export_summary()  导出语言维度汇总表
│   └── export_detail()   导出文件维度明细表
│
└── main()  主入口
    ├── 弹出文件夹选择对话框
    ├── 协调各模块执行完整流程
    └── 自动打开浏览器展示报告
```

设计说明：

- 采用分层架构，各层职责单一，新增语言或新增导出格式时仅需修改对应层，不影响其他模块
- 正则表达式统一预编译，避免逐行重复创建，提升大文件统计性能
- 所有模块通过标准化字典格式传递数据，耦合度低，便于单元测试

2.1.5 界面截图

程序最终输出为 HTML 可视化统计报告，界面分为五个核心区域：

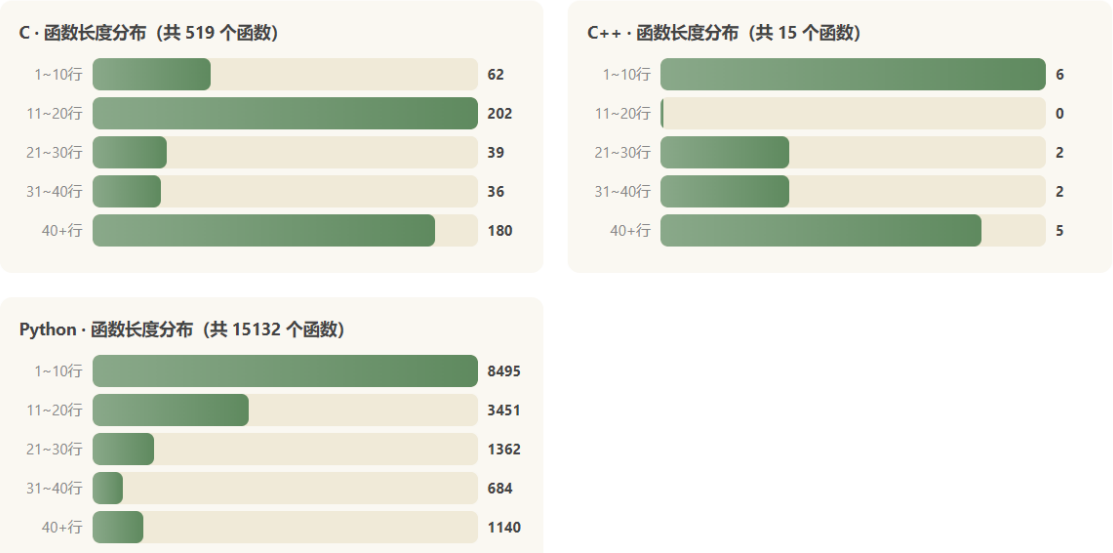
1. **概览卡片区**：展示源文件总数、代码行总数、注释行总数、函数总数四项核心指标，附带 CSV 文件下载入口
2. **语言对比区**：横向柱状图直观展示各语言代码行数占比
3. **详细统计表区**：完整展示各语言的文件数、代码行、空行、注释行、函数个数及函数长度四项指标
4. **函数长度分布区**：按行数分段统计函数数量分布，反映代码质量
5. **文件明细区**：可展开的文件级统计列表，展示每个文件的独立数据



详细统计表

语言	源文件数	代码行数	空行数	注释行数	函数个数	最大值	最小值	均值	中位数
C	229	135535	18672	33677	519	15148	2	164.5	19
C++	8	978	269	223	15	408	4	52.5	27
JAVA	0	0	0	0	0	-	-	-	-
Python	855	203744	46315	40020	15132	1414	1	16.4	9.0
C#	0	0	0	0	0	-	-	-	-

函数长度分布



文件级明细

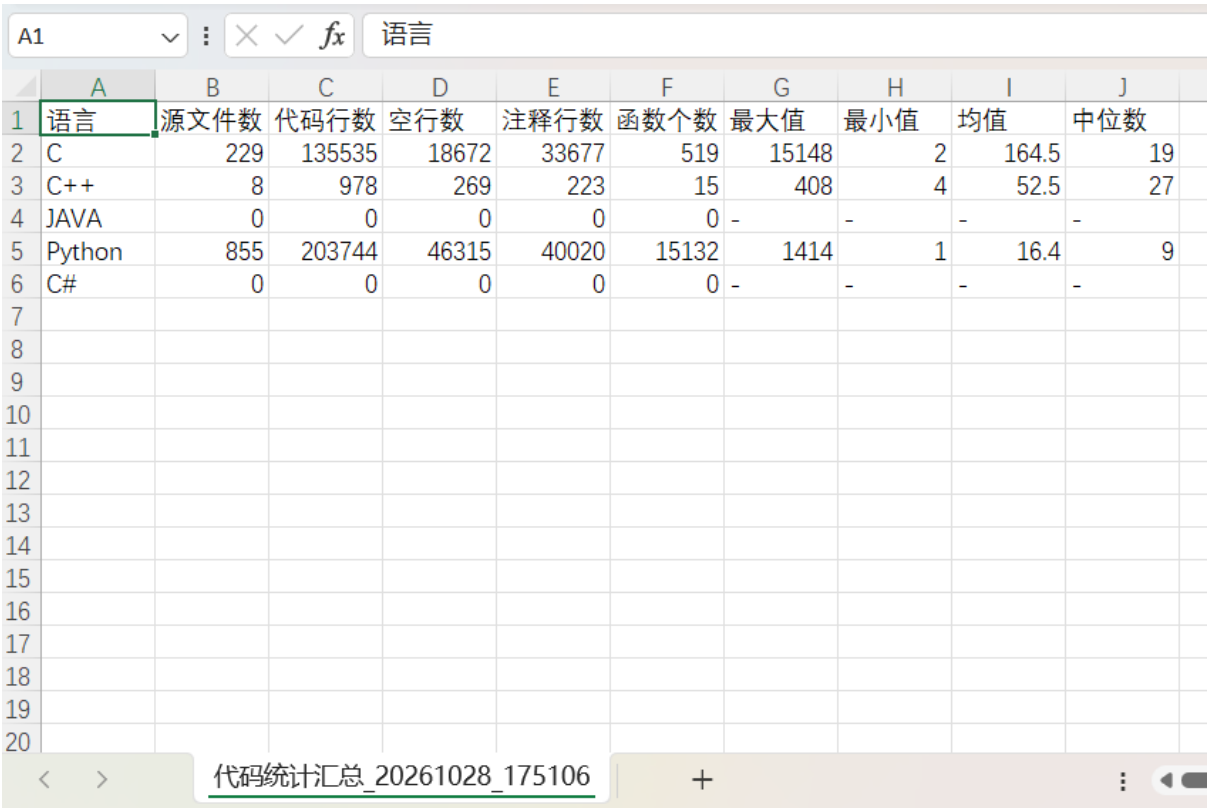
▶ C · 229 个文件

▶ C++ · 8 个文件

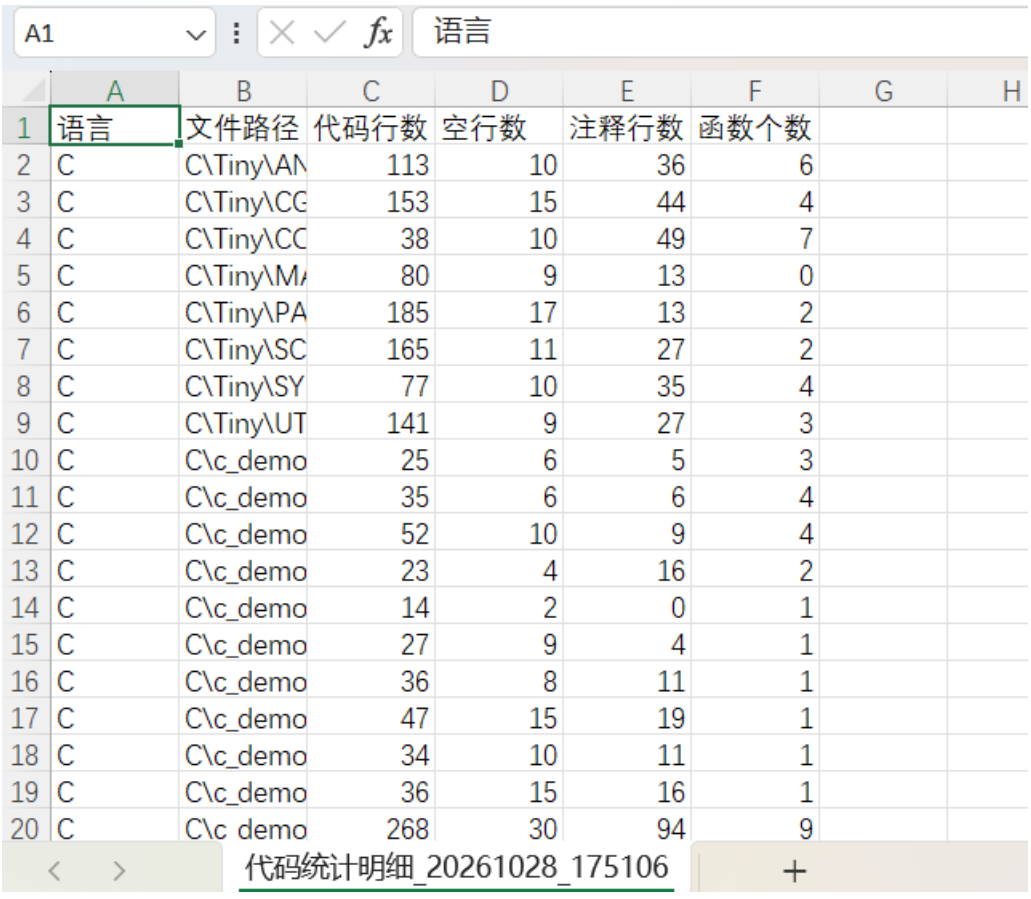
▶ Python · 855 个文件

统计口径说明

- 代码行数：去掉空行、纯注释行后的有效代码行。
- 空行数：空白行（无任何字符）。
- 注释行数：单行注释（//、#）及块注释（/* */、三引号 docstring）覆盖的行。
- 函数个数：按常见函数定义形式识别（Python 的 def；C 系语言的返回类型 + 函数名 + 括号）。
- 最大值/最小值/均值/中位数：统计每个函数的**行数**。



语言	源文件数	代码行数	空行数	注释行数	函数个数	最大值	最小值	均值	中位数
C	229	135535	18672	33677	519	15148	2	164.5	19
C++	8	978	269	223	15	408	4	52.5	27
JAVA	0	0	0	0	0	-	-	-	-
Python	855	203744	46315	40020	15132	1414	1	16.4	9
C#	0	0	0	0	0	-	-	-	-



语言	文件路径	代码行数	空行数	注释行数	函数个数
C	C\Tiny\AN	113	10	36	6
C	C\Tiny\CG	153	15	44	4
C	C\Tiny\CC	38	10	49	7
C	C\Tiny\MA	80	9	13	0
C	C\Tiny\PA	185	17	13	2
C	C\Tiny\SC	165	11	27	2
C	C\Tiny\SY	77	10	35	4
C	C\Tiny\UT	141	9	27	3
C	C\c_demo	25	6	5	3
C	C\c_demo	35	6	6	4
C	C\c_demo	52	10	9	4
C	C\c_demo	23	4	16	2
C	C\c_demo	14	2	0	1
C	C\c_demo	27	9	4	1
C	C\c_demo	36	8	11	1
C	C\c_demo	47	15	19	1
C	C\c_demo	34	10	11	1
C	C\c_demo	36	15	16	1
C	C\c_demo	268	30	94	9

2.2 功能 小学生口算出题

2.2.1 场景建模

用例：口算练习

参与者：学生

前置条件：网页小程序正常打开

简要说明：学生使用小程序完成口算相关练习、对战、查看激励、错题复盘等操作

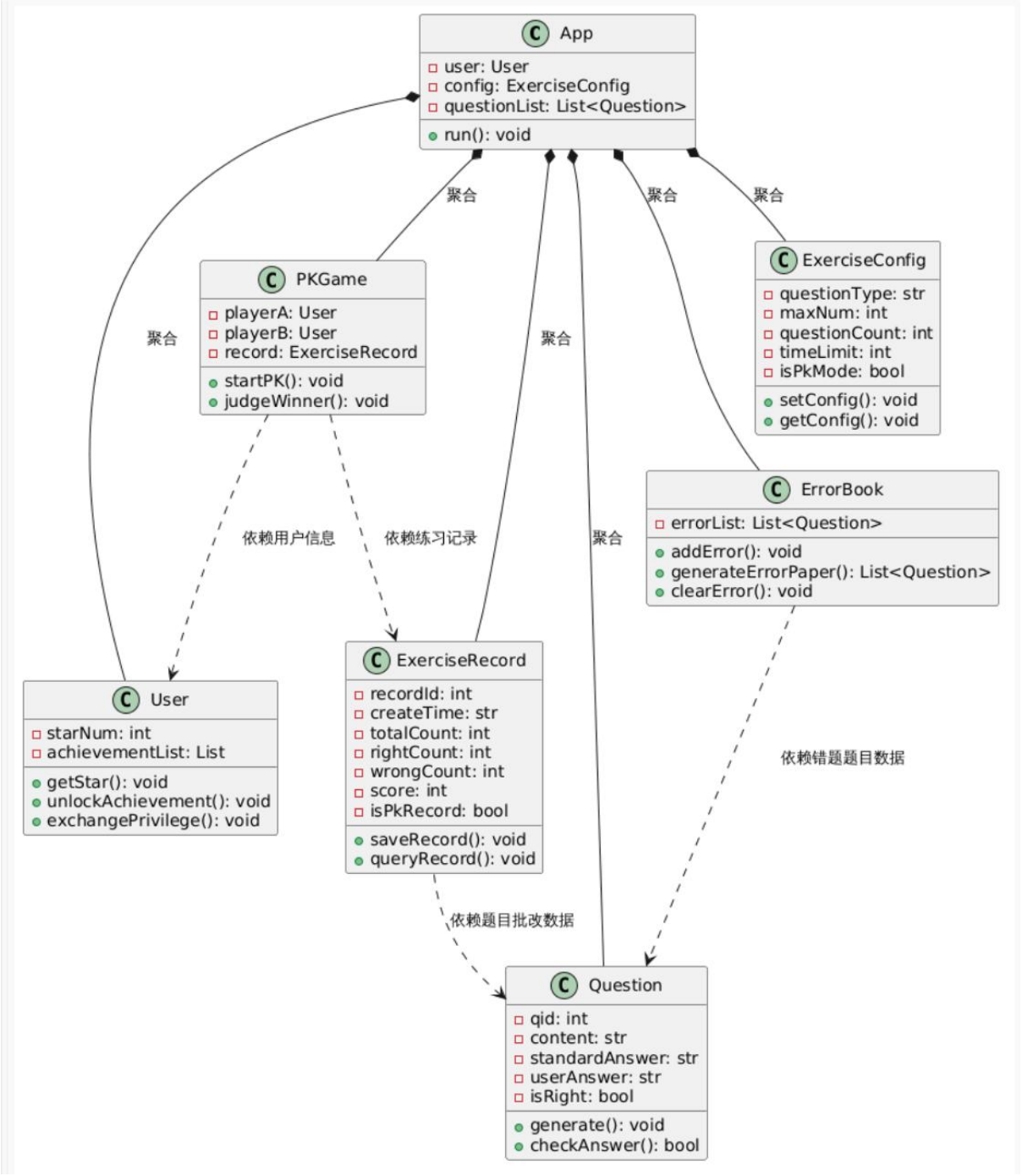
主流程

1. 学生配置题型、难度、题量等参数，选择普通练习或双人 PK 模式
2. 开始答题，系统计时、自动批改，对错实时标记
3. 练习完成后，获得星星，解锁成就；错题自动存入错题本，本次练习记录保存
4. 学生可打印试卷、查看练习历史、调取错题重做，也可进入商店使用星星兑换界面特权
5. 后置条件：练习数据、错题、星星、成就配置本地保存
6. 扩展分支

- 分支 1：选择双人 PK 模式，填写玩家名字，对战结束后系统判定胜负并发放奖励
- 分支 2：进入错题本，基于历史错题生成错题卷，再次练习
- 分支 3：进入成就商店，消耗星星兑换显示样式特权

2.2.2 类建模

采用面向对象思想进行类设计，遵循单一职责原则，共设计 **7 个核心类**，通过聚合与依赖关系协作完成全流程。



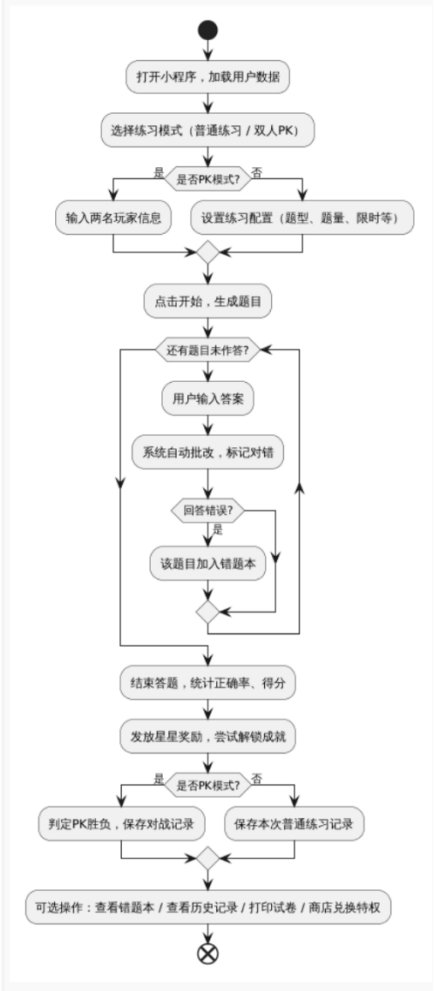
聚合关系：App 作为主控制类，聚合了用户、练习配置、题目、练习记录、错题本、双人对战六个功能模块，整体与部分的生命周期相互独立。

依赖关系：练习记录模块、错题本模块依赖题目模块输出的批改结果；PK 对战模块依赖用户信息与练习记录结构，无直接属性关联。

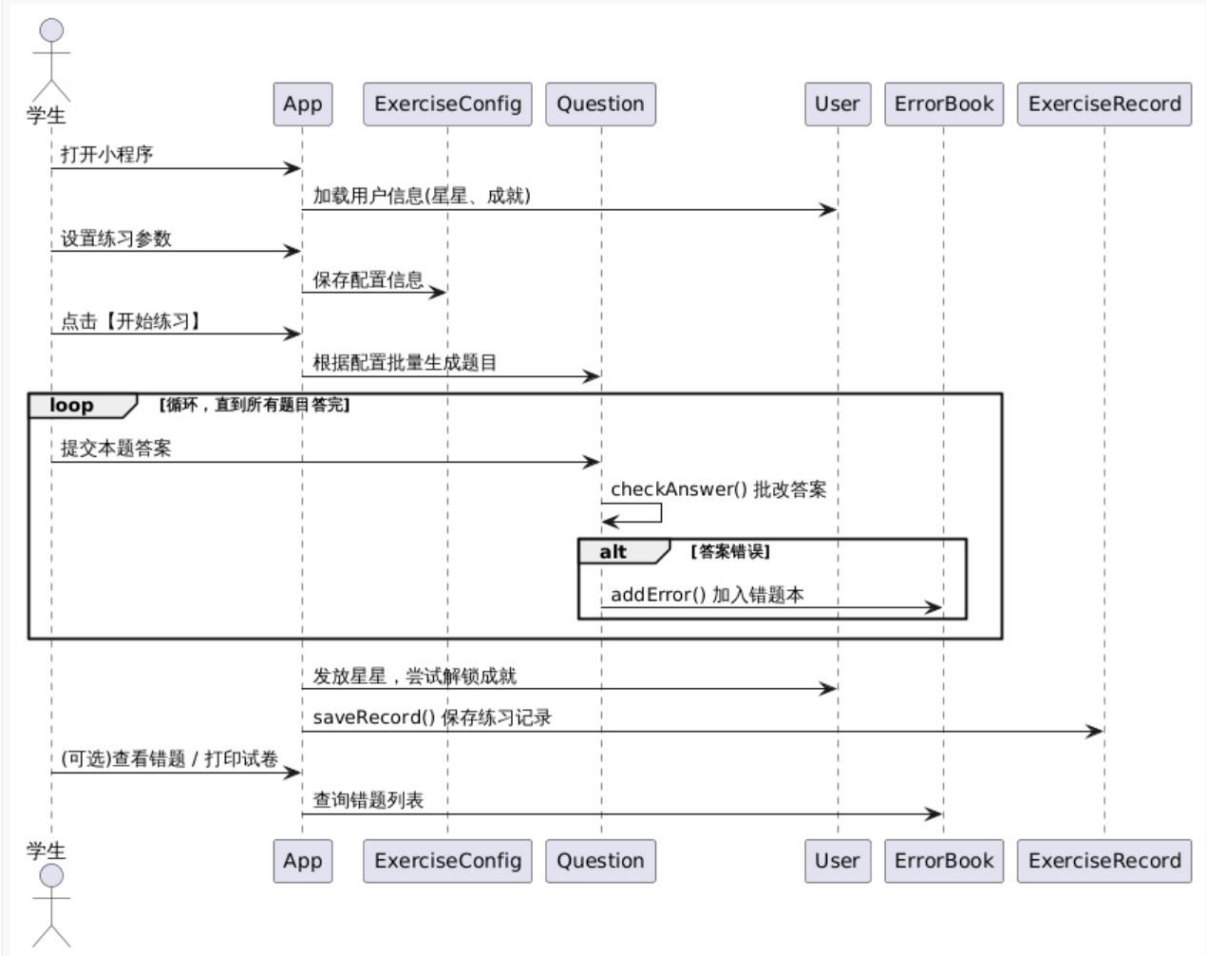
2.2.3 功能建模/行为建模

（1）活动图

描述小学生口算的完整执行流程，覆盖主事件流与分支判断



（2）顺序图 描述口算练习过程中，学生、主控 App 以及各个模块之间消息交互的时序过程。



2.2.4 代码结构及说明

项目采用单 HTML 文件模块化分层设计，核心结构如下：

index.html

├── 视图层（HTML）：页面布局、按钮、输入框、表格展示

├── 样式层（CSS）：页面样式、深色模式、打印样式

└── 逻辑层（JavaScript）

　　├── User 模块：用户星星、成就信息管理

　　├── ExerciseConfig 模块：练习参数配置

　　├── Question 模块：题目生成、答案批改

　　├── ExerciseRecord 模块：练习记录保存与查询

　　├── ErrorBook 模块：错题管理、错题卷生成

　　├── PKGame 模块：双人对战逻辑、胜负判定

　　└── App 主控模块：统一调度各个子模块，处理页面交互

设计说明：

本项目为网页版口算练习小程序，采用面向对象思想进行软件设计，整体遵循单一

职责原则，按照 **MVC 分层思路**划分视图、样式、业务逻辑三层。

视图层基于 HTML 实现，负责页面展示、接收用户操作，提供练习设置、答题输入、结果展示、错题查看、打印、PK 对战等交互入口；CSS 负责页面渲染，支持深色模式、打印适配，分离展示与业务逻辑。

业务逻辑层抽象出 7 个核心类：App、User、ExerciseConfig、Question、ExerciseRecord、ErrorBook、PKGame。其中 App 作为主控类，聚合其余所有业务模块，负责调度整个业务流程；User 管理用户星星、成就特权；ExerciseConfig 封装练习参数；Question 负责题目随机生成与答案批改；ExerciseRecord 持久化保存每一次练习数据；ErrorBook 实现错题收集、错题卷生成；PKGame 实现双人对战、胜负判定。模块之间使用聚合、依赖关系协作，低耦合高内聚。

行为建模使用活动图描述完整业务流程，区分普通练习、双人 PK 分支；顺序图刻画学生与各个模块之间消息交互时序，清晰展示出题、答题、批改、存错题、结算奖励的交互过程。

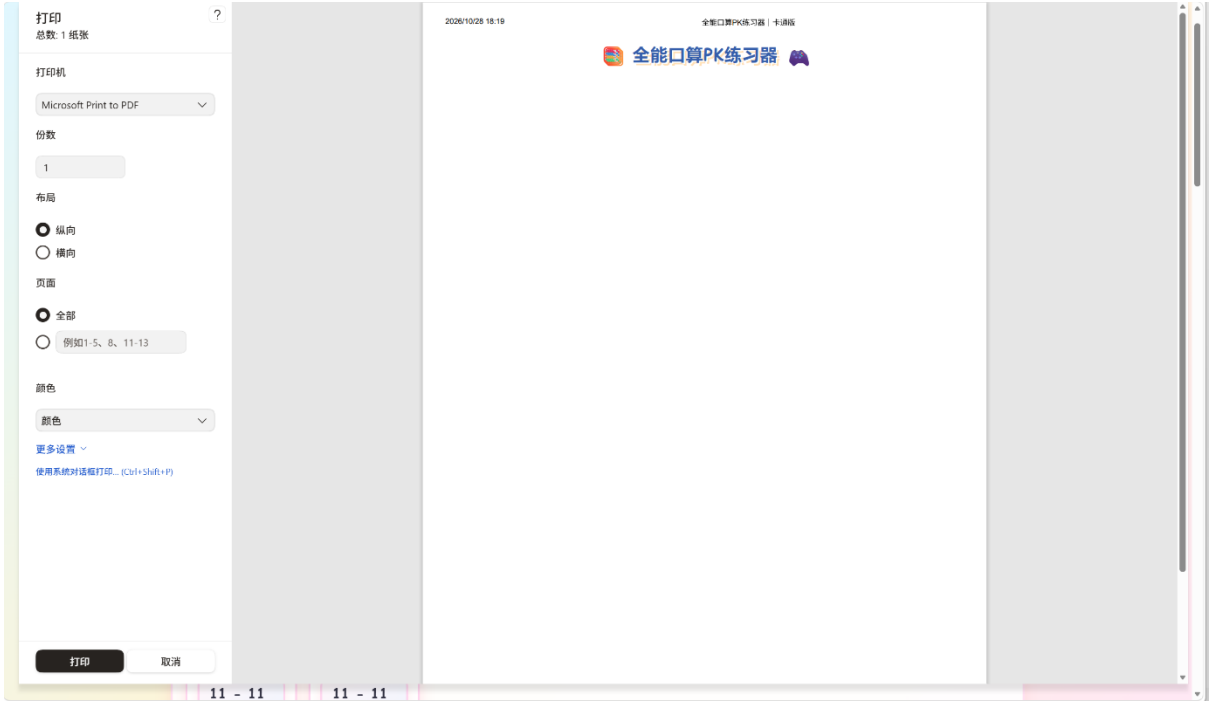
数据采用浏览器本地存储，关闭页面后用户星星、历史记录、错题数据不会丢失。系统支持自定义题型难度、自动批改、错题复盘、双人 PK、成就激励、试卷导出打印等功能，覆盖前期用例需求。

2.2.5 界面截图

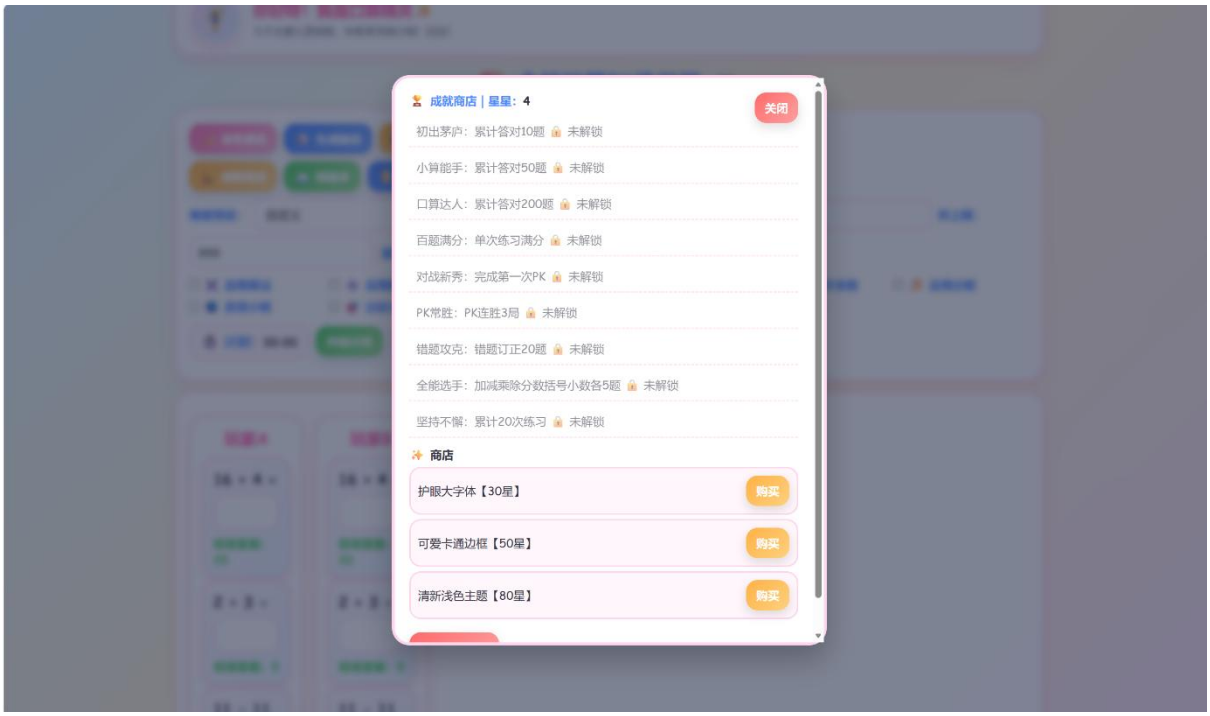
初始页面



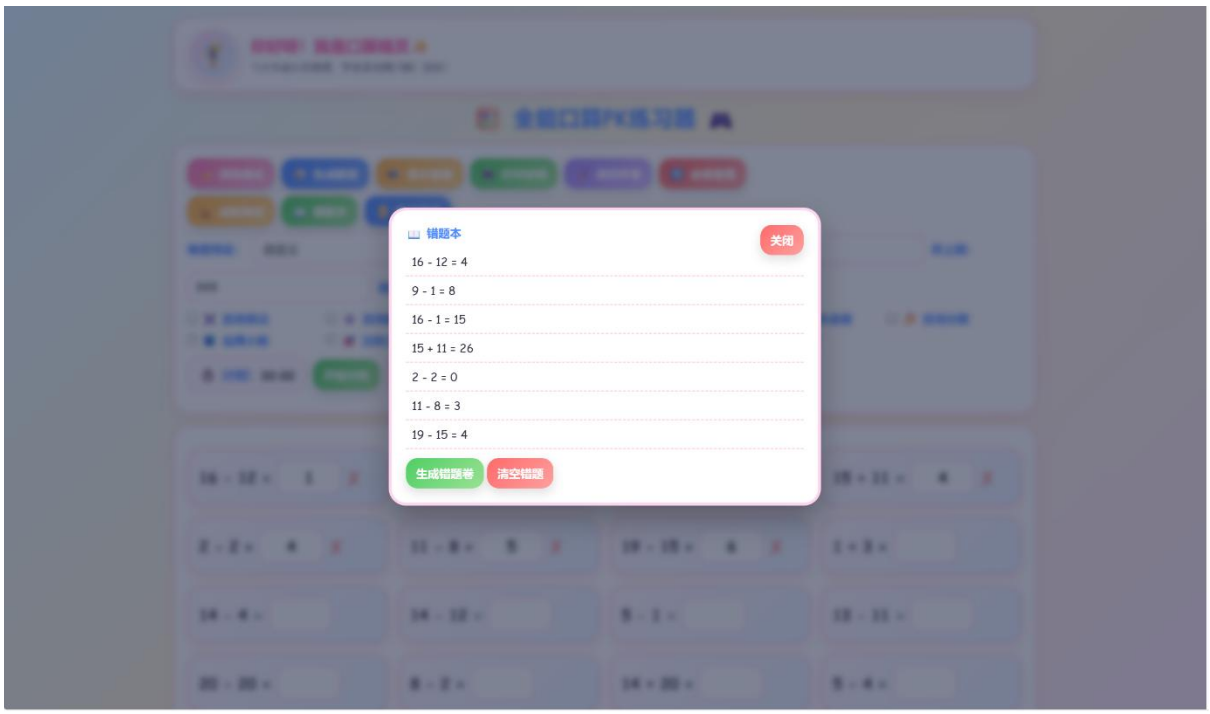
打 印 试 卷



成 就 商 店



错 题 本



2.3 功能 小组项目主页展示

小组项目主页网址：https://psycho933.github.io/software-project-homepage/

2.3.1 场景建模

用例名称： 小组项目主页浏览

参与者： 项目访客（授课老师、同学、其他开发者）

上下文目标： 访客通过公开网址访问小组项目主页，快速了解项目整体背景、核心功能、成员分工，并下载项目文档与工具资源。

前置条件：

1. 项目主页已部署至 GitHub Pages，可通过公开网址正常访问
2. 访客设备具备网络连接与主流浏览器环境
3. 仓库内相关文档、资源文件已上传至对应路径，链接有效

触发： 访客在浏览器中输入项目主页网址，或通过外部链接跳转进入主页

主事件流：

1. 访客访问主页网址，浏览器向 GitHub Pages 发起页面资源请求
2. 服务器返回 HTML、CSS 页面资源，浏览器完成页面渲染
3. 访客浏览页面头部与项目简介模块，了解项目背景与整体定位
4. 访客查看核心功能模块，了解四则运算出题、代码量统计两大模块的能力与特点

5. 访客查看小组成员模块，了解各成员的分工与负责模块
6. 访客点击文档下载或仓库链接，跳转至对应页面或触发文件下载
7. 访客完成信息获取，关闭页面或跳转至外部链接，用例结束

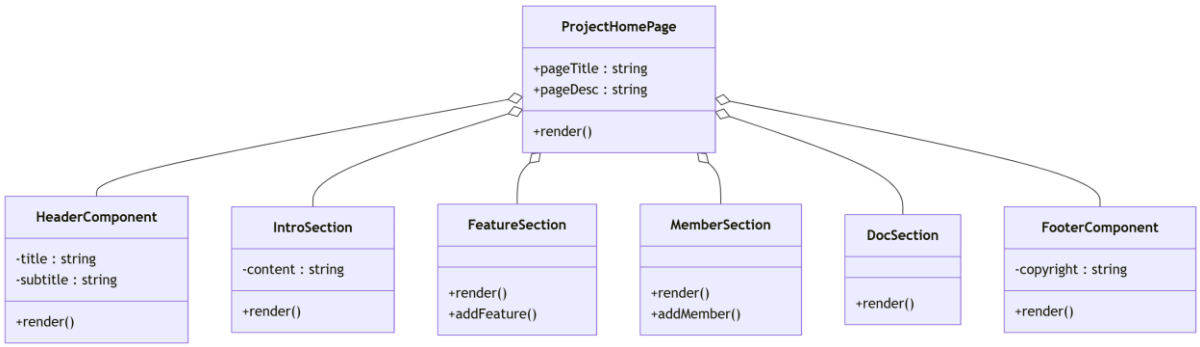
分支：

1. 网址输入错误或页面未部署完成：浏览器返回 404 错误，无法访问页面
2. 网络连接异常：页面加载失败，显示浏览器网络错误提示
3. 链接指向的文件不存在：点击后跳转至仓库 404 页面
4. 浏览器禁用 JavaScript：页面基础文本与布局正常展示，交互动效降级

频率： 课程作业评审、项目答辩阶段高频访问；日常开发维护阶段低频访问

2.3.2 类建模

采用前端组件化思想进行类设计，以页面主类聚合各功能模块组件，整体为静态页面结构，无后端业务类

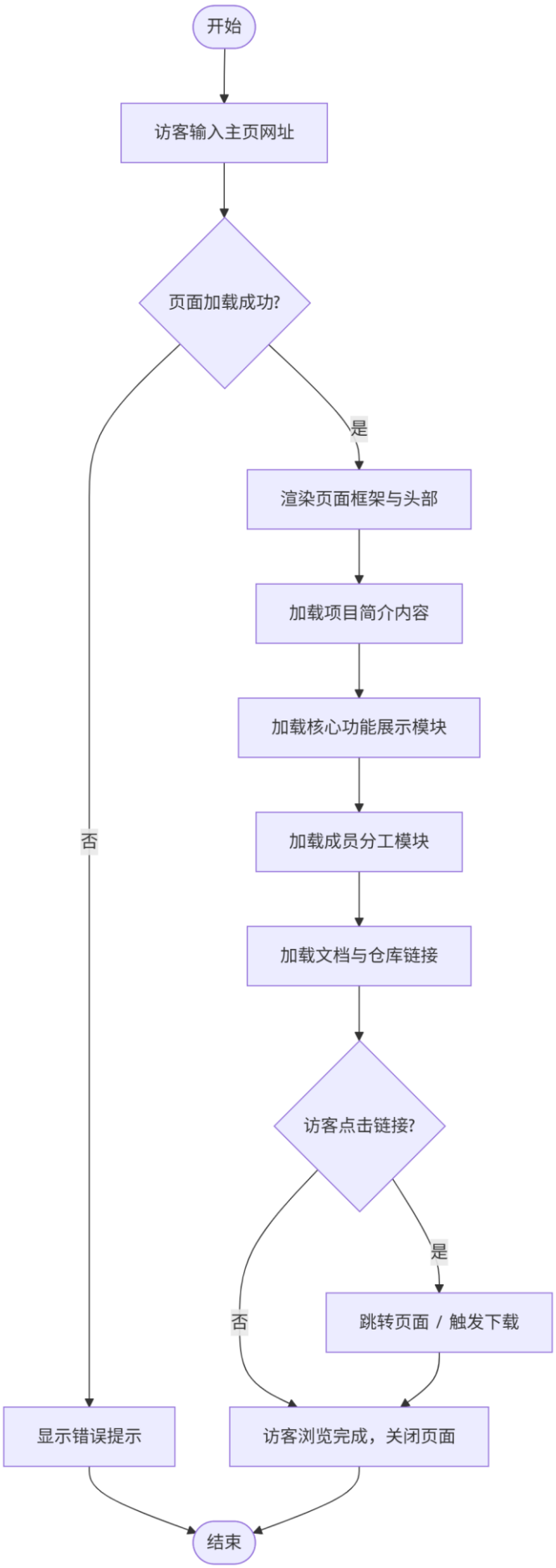


类关系说明：

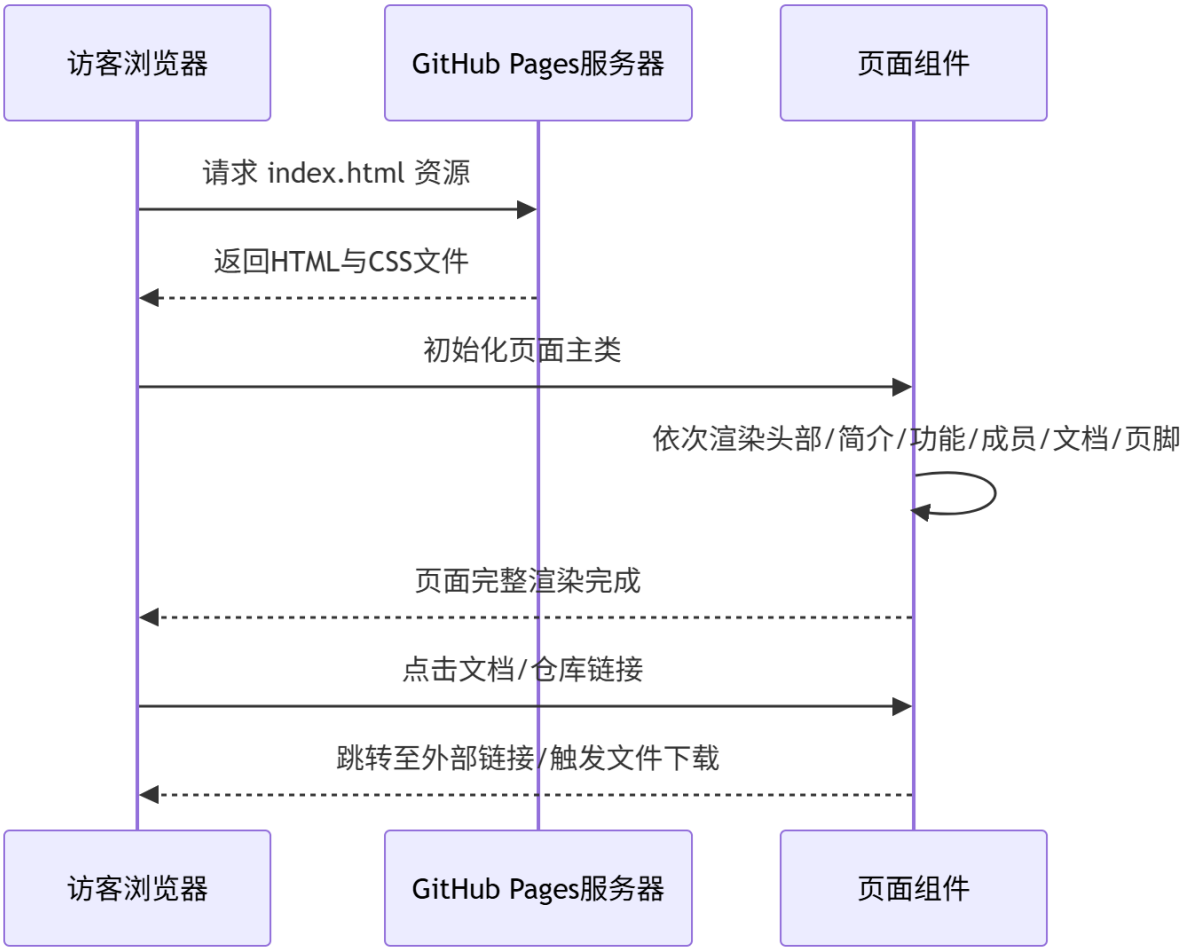
- **聚合关系：**ProjectHomePage 作为页面主类，聚合了头部、简介、功能、成员、文档、页脚 6 个组件类，各组件独立维护自身内容与渲染逻辑
- 所有组件均为展示型类，无复杂业务逻辑，仅负责数据渲染与页面呈现，符合静态主页的轻量化定位

2.3.3 功能建模/行为建模

（1）活动图（用户浏览主页流程）


开始
访客输入主页网址
页面加载成功?
是
否
渲染页面框架与头部
加载项目简介内容
加载核心功能展示模块
加载成员分工模块
加载文档与仓库链接
访客点击链接?
是
否
跳转页面 / 触发下载
显示错误提示
访客浏览完成，关闭页面
结束

（2）顺序图（页面加载与交互流程）



2.3.4 代码结构及说明

项目主页为单文件静态网页，采用 HTML + CSS 实现，无第三方依赖，核心结构如下：

index.html

```
├── <head> 头部
│   ├── meta 视口与编码配置
│   └── <style> 内嵌样式表
│       ├── 全局基础样式
│       ├── 容器与卡片布局样式
│       ├── 功能栅格布局样式
│       └── 按钮与页脚样式
│
└── <body> 页面主体
```

├── .header 页面头部标题区
├── .section 项目简介模块
├── .section 核心功能模块（双列卡片栅格）
├── .section 小组成员模块
├── .section 项目文档链接区
└── .footer 页脚版权区

设计说明：

- 采用单文件内嵌样式，部署简单，仅需一个文件即可完整运行
- 响应式布局，适配电脑与手机端浏览
- 纯静态无后端逻辑，依托 GitHub Pages 托管，零运维成本，适合课程项目展示

2.3.5 界面截图

主页整体采用卡片式布局，从上至下分为六个核心区域：

1. **头部标题区**：展示项目名称与课程归属信息
2. **项目简介区**：一句话说明项目背景与整体定位
3. **核心功能区**：双列卡片分别展示出题系统与代码统计工具，附带操作按钮
4. **成员分工区**：展示小组成员姓名与对应负责模块
5. **文档与仓库区**：提供项目的成果、期中报告下载入口与 GitHub 仓库跳转入口
6. **页脚区**：标注版权与部署方式



成员与分工

陈琦毅
负责：代码统计工具、项目主页创建

杨若随
负责：小学生口算出题项目

项目文档与工具

在线体验

🎮 四则运算出题系统（在线版）

工具与源码下载

📝 出题系统源码 (HTML)　📊 代码量统计工具 (Python)

课程文档

📄 中期报告 (PDF)

源码仓库

💻 GitHub 仓库